SIMULADOR DE FRONT-END DO CALORÍMETRO HADRÔNICO DO DETECTOR DO ATLAS NO LHC USANDO FPGA E PROCESSADOR ARM EMBARCADO

Thiago Campos Acácio Paschoalin[1,2] - thiago.paschoalin@cefetmg.br
Vinícius Pires Diniz de Oliveira[1] - vinicius.pires@cern.ch
Luciano de Manhães Andrade Filho[1] - luciano.andrade@ufjf.br
Diogo Alves Cardinot[3] - diogo.cardinot@iprj.uerj.br
Gustavo Barbosa Libotte[3] - gustavolibotte@iprj.uerj.br
[1]Universidade Federal de Juiz de Fora - UFJF
[2]Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG), Unidade Leopoldina, MG, Brasil
[3]Universidade do Estado do Rio de Janeiro, Instituto Politécnico - Nova Friburgo, RJ, Brazil

Resumo. *O contexto do futuro upgrade de alta luminosidade do Grande Colisor de Hádrons (LHC) impõe a necessidade de novos sistemas de aquisição de dados para o calorímetro de telhas (TileCal) do experimento ATLAS. Este trabalho apresenta um simulador de pulsos em tempo real, com parâmetros configuráveis em tempo real, projetado para validar novas técnicas de processamento de sinal. A plataforma foi implementada em um System-on-Chip (SoC), utilizando a lógica programável de um FPGA Cyclone V para a geração de pulsos a 40 MHz, em sincronia com as colisões do LHC, e um processador ARM dual-core para o controle. O processador ARM, executando um sistema Linux embarcado, permite a alteração dos parâmetros do simulador através de uma aplicação em C acessada remotamente via SSH. Os resultados, validados com a ferramenta Signal Tap Logic Analyzer, demonstram que parâmetros como a taxa de ocupação e o nível de offset são alterados em tempo real. A plataforma desenvolvida constitui uma ferramenta fundamental para o avanço da instrumentação eletrônica do TileCal, permitindo o desenvolvimento e teste de novas técnicas para aquisição de dados.*

Keywords: *Processamento em tempo real, Linux, FPGA, Física de Altas Energias*

1. INTRODUÇÃO

Um dos principais objetos de estudo da física de altas energias é o Grande Colisor de Hádrons (LHC - *Large Hadron Collider*), o maior e mais energético acelerador de partículas em funcionamento, gerenciado pela Organização Europeia para Pesquisa Nuclear - CERN (*Conseil Européen pour la Recherche Nucléaire*). Construído na fronteira entre França e Suíça, o acelerador de partículas consiste em um anel de 27 quilômetros de circunferência com ímãs supercondutores e diversas estruturas de aceleração para aumentar a energia das partículas ao longo do caminho.

No acelerador de partículas, dois feixes de prótons (de aproximadamente 323 bilhões de prótons) são acelerados a uma velocidade próxima a da luz e forçados a colidirem um contra o outro em pontos determinados do anel a uma taxa de 40 milhões de colisões por segundo. Os pontos de colisão estão localizados em 4 detectores distribuídos estrategicamente ao longo da circunferência do LHC. Estes detectores são: o ATLAS (***A T**horoida**L A**paratu**S***), CMS (***C**ompact **M**uon **S**olenoid*), ALICE (***A L**arge **I**on **C**ollider **E**xperiment*) e o LHCb (***L**arge **H**adron **C**ollider **B**eauty*). Eles têm particularidades em suas construções de acordo com o objetivo de estudo das colisões de cada um.

O experimento ATLAS publicou em 2012 o anúncio histórico do resultado da busca por uma das peças fundamentais do Modelo Padrão da Física de Partículas, o Bóson de Higgs: ”Estes resultados fornecem evidência conclusiva para a descoberta de uma nova partícula com massa de 126,0 ± 0,4 (est) ± 0,4 (sist) GeV. O parâmetro de força do sinal μ tem o valor de 1,4 ± 0,3 na massa ajustada, o que é consistente com a hipótese do Bóson de Higgs do Modelo Padrão μ = 1”ATLAS (2012). A declaração consolidou o LHC e o experimento ATLAS como uma das ferramentas mais precisas e poderosas para a ciência.

O sistema de calorimetria do detector do ATLAS é construído em camadas, na camada mais interna o Calorímetro Eletromagnético, sua principal função é medir a energia de elétrons e fótons. Externamente ao calorímetro eletromagnético está o Calorímetro de Telhas (TileCal) componente central do sistema de calorimetria hadrônica do ATLAS.

O TileCal é construído com placas de aço como material absorvedor alternadas com telhas cintilantes de plástico como material ativo. No momento em que ocorre uma colisão, as partículas podem se desintegrar formando novas partículas, ou desviar seu caminho do eixo do feixe de colisão. Para esses dois casos, a partícula, pós-colisão, deposita sua energia no calorímetro de telhas, que, por sua vez, emite uma quantidade de luz proporcional à energia da partícula.

A luz gerada pela telha cintilante é levada através de uma fibra óptica até fotomultiplicadores (PMTs) para transformar o sinal luminoso em sinal elétrico. Esse sinal elétrico é proporcional à energia da partícula e também de curta duração pela velocidade do efeito fotoelétrico, sendo possível considerá-lo como um impulso.

Para possibilitar a digitalização com circuitos sem grandes complexidades, é necessário que o sinal passe por um circuito de conformação que alonga o pulso no tempo, permitindo o correto funcionamento dos ADCs na frequência de 40 MHz, sem comprometer a qualidade da medição. Thiago C. A. Paschoalin (2025)

O calorímetro é dividido em 128 módulos e cada módulo é subdividido em células, onde cada célula possui um canal de leitura. Pela distância e o ângulo em relação ao ponto de colisão, é possível aferir uma característica energética em relação à ocupação - probabilidade de a célula capturar um sinal de energia de uma partícula proveniente da colisão.

O LHC passará por um processo planejado para aumentar sua luminosidade. Durante este período de parada programada para o upgrade, o detector estará inoperante, impossibilitando testes com dados reais. Nesse cenário, o desenvolvimento de simuladores em tempo real torna-se fundamental. A criação de uma plataforma de simulação, como a proposta, permite que o estudo e a validação de novas tecnologias e algoritmos para a eletrônica de aquisição de dados continuem de forma ininterrupta, garantindo que o sistema esteja preparado para as novas e mais exigentes condições de operação quando o acelerador for religado.

O simulador desenvolvido busca recriar com fidelidade sinais de energia captados pelo TileCal. É interessante o estudo da eletrônica para a conformação do sinal, bem como a sua digitalização e, por fim, a reconstrução do sinal de energia original da partícula. Gerar sinais de

energia em tempo real, com a característica energética do ambiente de estudo, é fundamental para a pesquisa de técnicas de processamento de sinais.

1.1 SIMULADOR DE PULSOS

O desenvolvimento do simulador *free-running* em primeiro momento ocorreu em MATLAB, partindo de três condições fundamentais. A definição do momento dos eventos de interesse, associados à ocupação em cada célula, o sorteio dos valores de energia para cada uma das amostras dos eventos de colisão e, finalmente, montar a característica energética de cada célula de acordo com a distribuição da energia mapeada. Vinicius P. D. de Oliveira (2025)

Os eventos de colisão têm uma característica específica sobre sua distribuição: precisam ser independentes entre si. Eventos correlatos podem gerar erros na análise das técnicas para processamento do sinal. Por esse motivo, foi necessário um estudo sobre algoritmos de geração de números aleatórios, importante na decisão se uma amostra será um evento de interesse ou não, e também no sorteio dos valores de energia, que representam a amplitude do pulso.

Usando uma sequência aleatória e definida a taxa de ocupação, pode-se obter uma sequência de valores zeros e uns, na qual o valor um representa um evento de interesse e o valor zero representa um não-evento. O valor sorteado na sequência aleatória, sendo menor que o valor da taxa de ocupação, é normalizado para 1, e o valor sorteado, maior do que a taxa de ocupação, é definido como 0. Dessa forma, é gerada uma sequência com a quantidade de eventos baseada na taxa de ocupação de cada célula.

Para o cálculo do valor da amplitude do pulso, utilizando uma Função de Distribuição Cumulativa (CDF - *Cumulative Distribution Function*) baseada no histograma de característica energética de cada célula, somam-se os valores de cada barra de forma sequencial. Após o somatório, é necessário fazer uma interpolação para se obter uma função que relacione o valor uniformemente distribuído e normalizado de 0 a 1 com um valor de energia. Essa etapa expõe a probabilidade de um valor de energia ocorrer.

Para a implementação prática, a função contínua da CDF é discretizada e armazenada em memória como uma Tabela de Consulta. A precisão com que a distribuição de energia original é replicada pode ser ajustada pelo tamanho desta tabela, que no simulador pode ser configurada para conter 128, 256, 512, 1024, 2048 ou 4096 valores de energia. Dessa forma, ao sortear os valores aleatórios, a sequência de amplitudes de pulso gerada pelo simulador reproduz o espectro de energia mapeado para a célula em análise.

Para validar novas técnicas do processamento de sinal é necessário um simulador capaz de operar em tempo real. A implementação em FPGAs (*Field-Programmable Gate Arrays*) permite o processamento em paralelo de alto desempenho já desenvolvido e testado por Fábio C. Luna (2024). Modelando toda a cadeia de captura e leitura do sinal do TileCal, gerando dados a 40MHz em sincronia com as colisões do LHC.

No Simulador de Fábio C. Luna (2024), é realizada uma convolução do sinal gerado como amplitude do pulso, por um filtro de Resposta ao Impulso Finita (FIR). Os coeficientes do filtro são valores amostrados e quantizados do pulso característico. Os testes mostraram que o simulador implementado é altamente eficiente, ocupando apenas 3% dos recursos lógicos do FPGA e operando a uma frequência máxima de 100.96 MHz, bem acima dos 40 MHz necessários.

Uma limitação da implementação inicial do simulador em hardware era a natureza estática de seus parâmetros. Características como o espectro de energia e a taxa de ocupação da célula eram fixas e não podiam ser alteradas em tempo real durante a geração dos pulsos. Para

superar essa restrição, a arquitetura do simulador foi portada para um *System-on-Chip* (SoC), um dispositivo que integra em um único chipset um processador FPGA e um processador ARM operando em paralelo.

Dessa forma, as tarefas foram otimizadas, o FPGA continua responsável pela geração dos pulsos em alta velocidade e o processador ARM, por sua vez, atua como um supervisor. Para gerenciar essa função, foi implementado um sistema operacional Linux embarcado no ARM, criando uma interface de controle que permite modificar online os parâmetros de operação do simulador, como a taxa de ocupação e o nível de *Offset*, atendendo à necessidade de ajustes dinâmicos durante a execução.

2. INTEGRAÇÃO FPGA E ARM

O simulador foi implementado no kit de desenvolvimento DE10-Nano da Terasic. Esta plataforma, centrada no System-on-Chip (SoC) Altera Cyclone V SE 5CSEBA6U23I7, integra em um único dispositivo um processador de propósito geral FPGA. Essa arquitetura permite uma divisão de trabalho otimizada, na qual o FPGA executa os algoritmos de simulação que exigem alta velocidade e paralelismo, enquanto o processador ARM gerencia a interface com o usuário e o controle dinâmico dos parâmetros da simulação.

O SoC Cyclone V combina um sistema de processador rígido (HPS), baseado em um processador dual-core ARM Cortex-A9 MPCore com clock de 800 MHz, e um tecido lógico de FPGA com 110.000 elementos lógicos (LEs). A placa DE10-Nano complementa essa estrutura com periféricos essenciais, incluindo 1 GB de memória RAM DDR3 conectada diretamente ao HPS para a execução de sistemas operacionais como o Linux, e um soquete para cartão MicroSD para armazenamento e boot do sistema. Além disso, a placa oferece uma porta Gigabit Ethernet para comunicação, uma porta USB 2.0 OTG e um acelerômetro de 3 eixos.

A principal vantagem desta arquitetura SoC é a comunicação de alta largura de banda e baixa latência entre o HPS e o FPGA, realizada através de pontes (bridges) de barramento AXI. As pontes ligando HPS-para-FPGA e FPGA-para-HPS são fundamentais para a aplicação, pois permitem que o sistema Linux rodando no ARM atue como mestre, acessando e modificando em tempo real os registradores e periféricos implementados no FPGA, e recebendo os resultados da simulação. É por meio desta ponte que os parâmetros do simulador, como taxa de ocupação e nível de offset, podem ser ajustados em tempo real.

Para viabilizar essa interação, o kit DE10-Nano utiliza uma imagem Linux pré-configurada, fornecida pelo fabricante e armazenada no cartão MicroSD. Ao inicializar, o HPS realiza o boot a partir deste cartão, carregando um sistema operacional completo que permite ao desenvolvedor interagir com o processador ARM através de uma interface de linha de comando (terminal). Esse ambiente Linux funcional oferece todas as ferramentas de um sistema, como a capacidade de executar scripts e compilar programas, para gerenciar a aplicação e enviar comandos para a lógica do FPGA.

O envio de comandos da linha de comando no Linux para o FPGA é fundamentado na arquitetura de memória mapeada do SoC. Do ponto de vista do sistema operacional, os registradores e periféricos implementados no FPGA são acessíveis como se fossem endereços na memória física do HPS. Portanto, para controlar o simulador, é necessário um programa que possa escrever valores em endereços físicos específicos, que são então transferidos para o FPGA através das pontes AXI. Um comando, como a alteração de um parâmetro de ocupação, é efetivamente a escrita de um valor numérico em um desses endereços de hardware pré-definidos.

A implementação prática dessa comunicação é realizada por meio de um programa em C, que opera no espaço de usuário (user-space) e pode ser executado diretamente do terminal. Esse programa obtém acesso direto ao hardware ao abrir o arquivo de dispositivo especial `/dev/mem`, que representa a memória física do sistema. Utilizando a chamada de sistema `mmap()`, ele mapeia o endereço físico do registrador alvo no FPGA para o espaço de endereçamento virtual do programa. Uma vez mapeado, o programa pode tratar esse endereço como um simples ponteiro em C; a escrita de um valor nesse ponteiro resulta em uma escrita direta no registrador do FPGA, permitindo o controle dinâmico da simulação a partir de um software.

3. CONFIGURANDO O SIMULADOR

Primeiramente, uma pequena inclusão no simulador desenvolvido por Fábio C. Luna (2024) foi realizada para suportar a adição de um nível de compensação na saída do conformador de pulsos. Esse nível de compensação é conhecido no TileCal como *Offset* e ele tem a função de prover apenas sinais de tensão positivos para o Conversor Analógico-Digital (ADC), sendo um valor que pode ser alterado pelo operador durante as colisões. Neste trabalho, a simulação do ADC ainda não foi implementada, mas o sinal de *Offset* já foi incluído como preparação para uma versão atualizada do simulador. Essa mudança consiste em apenas uma adição simples ao sinal de saída do conformador de pulso, como demonstrado na Figura 1, com o nível de *Offset* sendo configurado também pelo Linux.

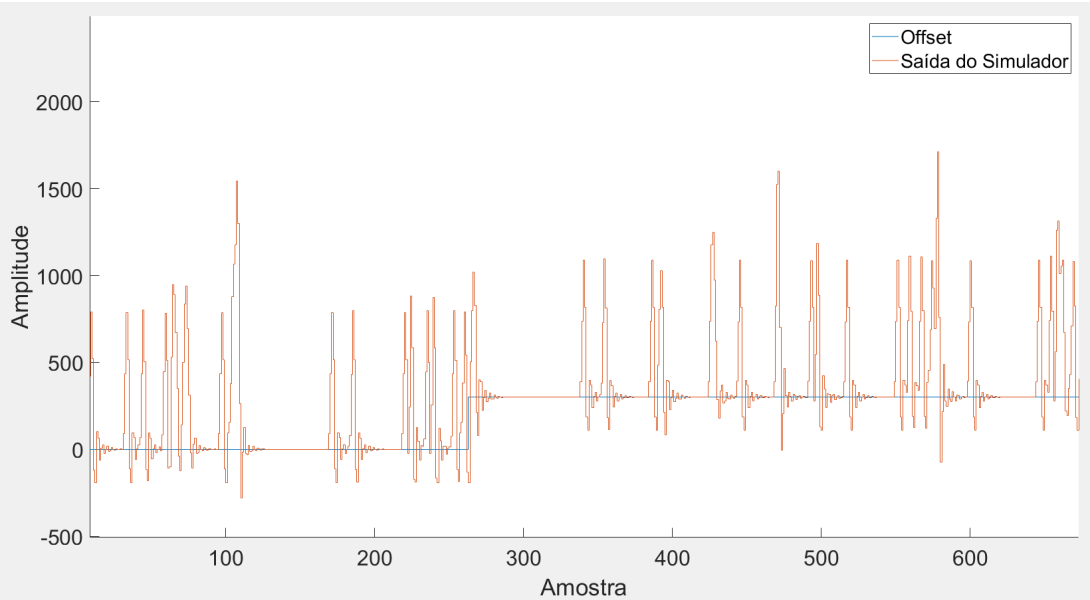

Figura 1: Variação do valor de `offset` em comparação a saída do simulador.

Com essa alteração explicitada, será mostrado como configurar o FPGA para se comunicar com o HPS.

3.1 CONFIGURANDO O FPGA

É disponibilizado pela Terasic (fabricante do kit DE10-Nano-Soc) um projeto padrão para ser utilizado como base para a utilização do FPGA em conjunto com o HPS. Neste, as

instâncias necessárias já estão presentes e, principalmente, possui o arquivo de configuração do QSYS/Plataform Designer (subprograma presente na IDE do Quartus) pré-configurado. Esse arquivo citado necessita de pequenas modificações para atender à proposta deste trabalho. Através dessa plataforma será gerado o código HDL (Hardware Description Language) do sistema principal.

A comunicação entre o HPS e o FPGA, é realizada através das pontes AXI, que foram configuradas para otimizar tanto a taxa de transferência quanto o controle. As pontes de maior largura de banda, HPS-para-FPGA e FPGA-para-HPS, foram configuradas com 128 bits, sendo ideais para transferências de grandes volumes de dados. Em contrapartida, a ponte Lightweight HPS-to-FPGA, configurada com 32 bits, é projetada para o acesso a registradores com latência mínima, sendo a via utilizada pelo ARM para enviar os comandos de ajuste dos parâmetros do simulador para a lógica em tempo real no FPGA.

Configura-se uma porta paralela de entrada e saída (PIO) de 7 bits para a ocupação, enquanto outra porta PIO é utilizada para o sinal de *offset*, mas com 13 bits. Os sinais de *clock* e *reset* são conectados a estas portas, exportando a saída do bloco para que o FPGA tenha acesso aos valores dados.

No *top level* do projeto padrão é possível incluir as duas portas de saída da PIO (ocupação e *Offset*) para utilização no simulador. O simulador agora é encapsulado em uma instância para uma melhor organização do projeto, como demonstrado na Figura 2. Os sinais então de ocupação e *Offset* entram no bloco do simulador nas portas destinadas a eles, onde todo o processo se dá normalmente como descrito em Fábio C. Luna (2024). A partir de então, toda alteração no valor das duas variáveis indicadas é atribuída automaticamente ao simulador de pulsos.

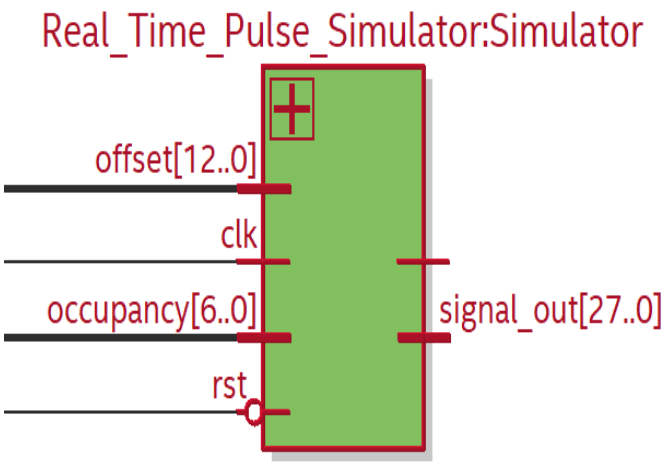

Figura 2: Instância do Simulador no RTL *Viewer*

Por fim, é necessário configurar o Signal Tap Logic Analyser para conseguir avaliar o funcionamento em tempo real dessas mudanças. O Signal Tap é uma ferramenta presente na IDE do Quartus que permite, através do JTAG, a inspeção do valor dos sinais internos dos blocos presentes no projeto, semelhante a um analisador lógico. Configura-se então os sinais que se desejam inspecionar, como a ocupação, *Offset*, sinal final da simulação calorímetro , dentre outros.

Após a geração e compilação do HDL no Quartus Prime, o passo seguinte é a implantação

do design no hardware do FPGA, processo que pode ser realizado através de dois métodos. Em uma via, utiliza-se a programação via JTAG, na qual o arquivo de configuração .sof (SRAM Object File) é transferido diretamente para o FPGA através da interface USB-Blaster II integrada na placa. Esta abordagem é rápida, porém volátil, o que significa que a configuração é perdida quando a placa é desligada. No outro caminho, o arquivo .sof é convertido para o formato .rbf (Raw Binary File) e inserido na raiz do cartão MicroSD que contém a imagem Linux. Dessa maneira, o bootloader executado pelo processador ARM se encarrega de ler este arquivo e programar o FPGA automaticamente.

Terminado o processo de configuração do FPGA necessário para funcionamento, a próxima etapa consiste na configuração do Linux e dos programas auxiliares necessários para atender ao propósito.

3.2 CONFIGURANDO O LINUX

O primeiro passo para a preparação do Kit de Desenvolvimento é a criação de um cartão MicroSD inicializável com a imagem Linux customizada para a placa. A imagem fornecida pelo fabricante deve ser gravada no cartão MicroSD. Este cartão conterá não apenas o kernel Linux e o sistema de arquivos, mas também o bootloader, que é o primeiro software a ser executado pelo processador ARM. Outras distribuições Linux podem ser usadas para a utilização da DE10-Nano-SoC, concebidas para esse propósito.

Com o cartão SD preparado, a próxima etapa é configurar corretamente o hardware da placa para garantir que a inicialização ocorra a partir do HPS. Isso é feito através das chaves MSEL (SW10), que devem ser ajustadas para o modo ”FPGA configured from HPS”(configuração 01010). Essa configuração instrui o SoC a carregar e executar o bootloader a partir do cartão SD, que, por sua vez, se encarregará de programar o tecido lógico do FPGA com o design do simulador.

O passo seguinte é o desenvolvimento do programa de controle em C que rodará no ambiente Linux. O código-fonte é escrito em um computador host e, para ser executado no processador ARM da placa, ele deve ser compilado utilizando uma cadeia de ferramentas de compilação cruzada, como a arm-linux-gnueabihf-gcc, que é fornecida pelo pacote Intel SoC EDS (Embedded Design Suite). Para que o programa em C possa interagir com a lógica do FPGA, ele precisa conhecer os endereços de memória dos periféricos (como os PIOs) implementados no Platform Designer. Essa informação é obtida gerando-se um arquivo de cabeçalho (hps_0.h) a partir do projeto Quartus. Ferramentas fornecidas pela Intel, como o script generate_hps_qsys_header.sh, extraem o mapa de endereços do sistema Qsys e o convertem em constantes `#define` que podem ser incluídas diretamente no código C.

Finalmente, o design de hardware compilado do simulador deve ser carregado no FPGA. Para a operação autônoma, o arquivo .sof gerado pelo Quartus é convertido para o formato .rbf (Raw Binary File) e copiado para a raiz do cartão SD. Durante a inicialização, o bootloader lê este arquivo e programa o FPGA automaticamente. Para fins de depuração, a programação pode ser feita de forma volátil via JTAG, utilizando o arquivo .sof e a ferramenta Quartus Programmer, o que permite carregar novos designs sem a necessidade de reiniciar a placa ou modificar o cartão SD.

A interação com o sistema operacional Linux, executado no HPS da placa DE10-Nano, pode ser realizada através de três abordagens distintas. O manual do kit sugere o uso de um emulador de terminal, o PuTTY, descrevendo especificamente sua utilização para estabelecer uma conexão serial através da interface UART-para-USB integrada. Este método é ideal para a

configuração inicial e depuração, pois não requer uma conexão de rede. Adicionalmente, para acesso remoto, a porta Gigabit Ethernet da placa permite a comunicação via SSH (Secure Shell), um protocolo também suportado por ferramentas como o PuTTY, viabilizando uma interface de linha de comando segura através da rede. Por fim, é possível operar a placa de forma autônoma, como uma estação de trabalho independente, conectando-se diretamente a um teclado à porta USB do HPS e um monitor à saída HDMI, que é controlada pelo FPGA.

4. RESULTADOS

A interface de controle para a alteração dos parâmetros do simulador é implementada através de uma aplicação em C, executada no sistema Linux embarcado no HPS. O programa, intitulado *change_occupancy*, é acessado remotamente por meio de uma sessão SSH, oferecendo ao usuário uma interface de linha de comando para a interação em tempo real com o hardware. Ao ser executada, a aplicação apresenta um menu interativo que permite selecionar qual parâmetro da simulação se deseja modificar, como a taxa de ocupação ou o valor de *Offset*. Após a escolha do usuário e a inserção do novo valor, o programa se encarrega de realizar a escrita no registrador correspondente no FPGA. A Figura 3 ilustra uma sessão de exemplo, na qual a taxa de ocupação do simulador é alterada para o valor 80.

```
=== MENU ===
1 - Set occupancy (0 to 127)
2 - Set offset (-4096 to 4095)
0 - Exit
Select an option: 1
Enter occupancy (0 - 127): 80
Occupancy updated to 80
```

Figura 3: Interface de linha de comando da aplicação `./change_occupancy`.

As figuras a seguir apresentam os resultados obtidos através da ferramenta Signal Tap, onde os sinais de entrada *occupancy* e *offset* são controlados via SSH, e o sinal de saída hits representa os pulsos de energia gerados.

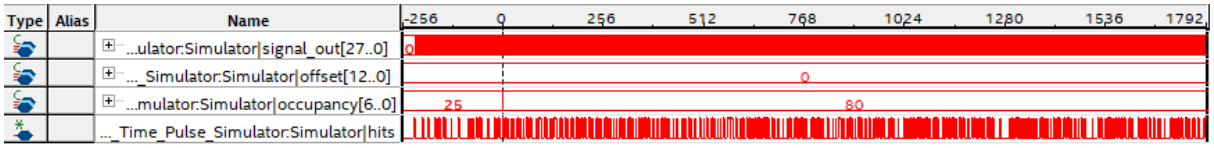

Figura 4: Signal Tap mostrando a alteração do parâmetro `occupancy` (de 25 para 80)

A Figura 4 demonstra o controle dinâmico da taxa de ocupação. Inicialmente, o parâmetro *occupancy* é configurado com um valor baixo (25 de 127). Como resultado, o sinal de saída hits exibe uma sequência de pulsos esparsos, indicando uma baixa probabilidade de ocorrência de eventos. Em dado momento, um comando é enviado pelo HPS para alterar o valor de occupancy para 80. A densidade de pulsos no sinal hits aumenta, correspondendo à nova e maior taxa de ocupação.

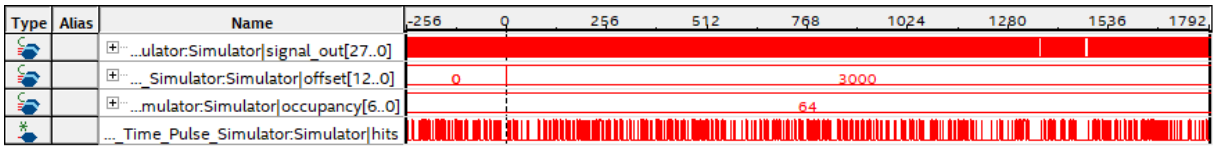

Figura 5: Signal Tap mostrando a alteração do parâmetro `offset` (de 0 para 3000)

A Figura 5 demonstra o controle de um segundo parâmetro, o offset. Nesta captura, o valor do *offset* é alterado de 0 para 3000 através de um comando via SSH. Embora o efeito no sinal hits não seja visualmente direto como o da ocupação, a alteração do valor no registrador interno do FPGA confirma que a interface de controle é capaz de modificar diferentes parâmetros da simulação enquanto ela está em plena execução.

O relatório de compilação, resumido na Figura 6, demonstra que a síntese, o *place-and-route* e a geração do arquivo de configuração foram concluídos com sucesso e valida a alta eficiência do simulador em termos de utilização de recursos de *hardware*.

Flow Summary

<<Filter>>

Flow Status	Successful - Fri Aug 22 14:46:19 2025
Quartus Prime Version	23.1std.1 Build 993 05/14/2024 SC Lite Edition
Revision Name	DE10_NANO_SoC_GHRD
Top-level Entity Name	DE10_NANO_SoC_GHRD
Family	Cyclone V
Device	5CSEBA6U23I7
Timing Models	Final
Logic utilization (in ALMs)	3,119 / 41,910 (7 %)
Total registers	5273
Total pins	172 / 314 (55 %)
Total virtual pins	0
Total block memory bits	102,912 / 5,662,720 (2 %)
Total DSP Blocks	12 / 112 (11 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1 / 6 (17 %)
Total DLLs	1 / 4 (25 %)

Figura 6: Relatório de compilação do projeto no Quartus Prime

A análise do relatório mostra que o design completo do simulador consome apenas 3.119 Módulos de Lógica Adaptativa (ALMs), o que corresponde a 7% do total disponível no dispositivo. O uso de recursos de memória embarcada também é mínimo, com 102.912 bits (apenas 2% do total). Para as operações de multiplicação, principalmente no filtro FIR, foram alocados 12 blocos DSP (11% do total). Notavelmente, apenas 1 dos 6 PLLs disponíveis (17%) foi necessário para gerar o clock de 40 MHz e sincronizar todo o sistema, sendo possível utilizar a frequência máxima de 78 MHz.

5. CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a validação de um simulador em tempo real para o sistema de leitura do calorímetro TileCal, implementado no Kit de Desenvolvimento DE10-Nano. A arquitetura híbrida, que combina um processador ARM executando Linux para

controle e um FPGA para processamento paralelo, demonstrou ser uma solução eficiente para emular o comportamento do detector. A capacidade de alterar dinamicamente parâmetros da simulação, como a taxa de ocupação e o nível de Offset, através de uma interface de linha de comando via SSH, foi validada com sucesso utilizando a ferramenta Signal Tap Logic Analyzer.

O simulador HITS, portanto, cumpre seu objetivo de ser uma ferramenta acessível permitindo pesquisas e desenvolvimentos de novas técnicas de processamento de sinais em um ambiente que replica as condições de tempo real do LHC, sem a complexidade dos frameworks de simulação padrão do CERN. Como trabalho futuro, propõe-se a inclusão da simulação do Conversor Analógico-Digital (ADC), o que aproximará ainda mais o sistema do comportamento completo do canal de leitura do TileCal e alteração do módulo do calorímetro a ser analisado.

Referências

ATLAS (2012). Observation of a new particle in the search for the standard model higgs boson with the atlas detector at the lhc. *Physics Letters B*.

Fábio C. Luna, Ualison F. Dias, P. H. B. L. T. C. A. P. T. M. Q. L. M. d. A. F. (2024). Real-time fpga-based simulator for the tile calorimeter readout system in the atlas experiment. *Encontro Nacional de Modelagem Computacional - ENMC*.

Thiago C. A. Paschoalin, Tiago M. Quirino, L. M. d. A. F. (2025). Implementação em fpga do circuito do condicionamento de sinal do calorímetro hadrônico do atlas. *Simpósio Brasileiro de Automação Inteligente - SBAI*.

Vinicius P. D. de Oliveira, Thiago C. A. Paschoalin, L. M. d. A. F. (2025). Simulador de pulsos para o upgrade do calorímetro hadrônico do experimento atlas no cern. *Simpósio Brasileiro de Automação Inteligente - SBAI*.

APPENDIX A

A REAL-TIME SIMULATOR FOR THE ATLAS CALORIMETER FRONT-END USING FPGA AND AN EMBEDDED ARM PROCESSOR

Abstract. *In the context of the future High-Luminosity upgrade of the Large Hadron Collider (LHC) requires new data acquisition systems for the Tile Calorimeter (TileCal) of the ATLAS experiment. This work presents a real-time pulse simulator with dynamically configurable parameters, designed to validate new signal processing techniques. The platform was implemented on a System-on-Chip (SoC), utilizing the programmable logic of a Cyclone V FPGA for pulse generation at 40 MHz, synchronized with the LHC collisions, and a dual-core ARM processor for control. The ARM processor, running an embedded Linux system, allows for the modification of the simulator's parameters through a C application accessed remotely via SSH. The results, validated with the Signal Tap Logic Analyzer tool, demonstrate that parameters such as the occupancy rate and offset level are changed in real-time. The developed platform constitutes a fundamental tool for the advancement of the TileCal's electronic instrumentation, enabling the development and testing of new data acquisition techniques.*

Keywords: *Real-time processing, Linux, High-energy physics, FPGA*